PROJETO ESG + DEVOPS

Cidades ESG Inteligentes

Órbita Alerta

Documentação técnica da entrega

Objetivo	Aplicar CI/CD, containerização e orquestração a uma API ESG executável.
Integrante	Micaela Guimarães — FIAP
Stack	C# / .NET 8, MongoDB, Docker Compose e GitHub Actions

Versão preparada em 27/09/2026

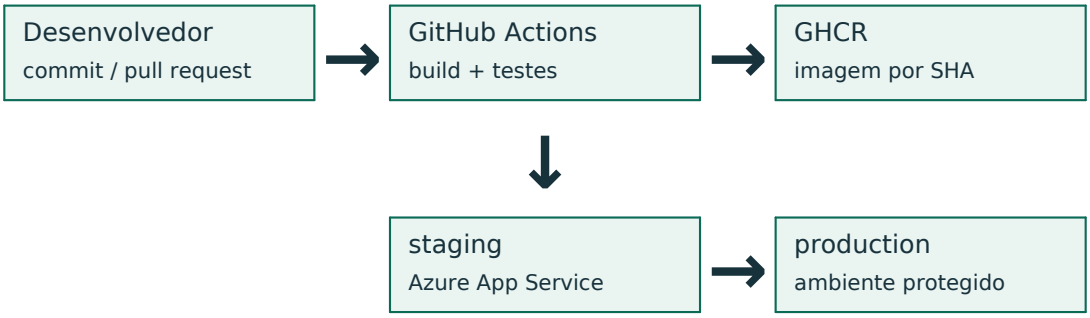
1. Contexto e objetivo

O Órbita Alerta é uma solução ESG voltada ao registro de alertas ambientais, áreas monitoradas e ocorrências de gestão. Nesta etapa, o projeto foi adaptado para representar um ciclo de entrega mais próximo de um ambiente produtivo: o código possui API executável, persistência em MongoDB, testes automatizados, imagem Docker e fluxo de promoção entre staging e produção.

A implementação foi preparada para execução real. As evidências de pipeline e de deploy externo não foram simuladas no documento: dependem da conexão do repositório às contas GitHub/Azure e da configuração dos secrets descritos no README.

2. Arquitetura da solução

A arquitetura separa a aplicação da persistência e utiliza o mesmo artefato de imagem no processo de promoção. No ambiente local, Docker Compose cria a API e o MongoDB em uma rede dedicada, com volumes nomeados para preservar os dados.



Componentes principais

• ASP.NET Core Minimal API em .NET 8, com endpoint de healthcheck e operações de consulta/criação de alertas.

• MongoDB 7, collection alerts e conexão configurável por variável de ambiente.

• Docker Compose com rede, volumes e healthcheck do banco.

• GitHub Actions para integração contínua, publicação no GHCR e deploy em dois ambientes.

3. Pipeline CI/CD

O workflow .github/workflows/ci-cd.yml é acionado em pull requests e pushes nas branches develop, qa e main. O pipeline foi dividido em jobs para deixar explícita a dependência entre qualidade, empacotamento e promoção.

Job	Quando	Responsabilidade
build-test	PR e push	Restore, build Release, testes xUnit e upload dos resultados.
container	push em qa/main	Login no GHCR, build da imagem e tags por SHA/branch.
deploy-staging	push em qa	Deploy automático no ambiente staging.

Job	Quando	Responsabilidade
deploy-production	push em main	Deploy no ambiente production, sujeito à aprovação configurada no GitHub.

Secrets previstos

Os nomes dos Web Apps e os publish profiles são mantidos fora do código. O pipeline espera AZURE_WEBAPP_STAGING, AZURE_WEBAPP_STAGING_PUBLISH_PROFILE, AZURE_WEBAPP_PRODUCTION e AZURE_WEBAPP_PRODUCTION_PUBLISH_PROFILE. As credenciais do MongoDB devem ser configuradas como variáveis do App Service ou secret manager equivalente.

4. Containerização e orquestração

O Dockerfile utiliza uma estratégia multi-stage. A etapa de build contém o SDK completo apenas durante a compilação; a etapa final utiliza a imagem menor de runtime do ASP.NET Core. O processo reduz a superfície da imagem e evita transportar artefatos de desenvolvimento para execução.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
RUN dotnet restore ...
RUN dotnet publish ... --configuration Release

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
EXPOSE 8080
USER app
ENTRYPOINT ["dotnet", "OrbitaAlerta.Api.dll"]
```

O Compose define API e MongoDB, configura a dependência pelo healthcheck e monta os volumes mongodb-data e mongodb-config. A rede orbita-alerta-network mantém a comunicação dos serviços isolada do restante do host.

```
cp .env.example .env
docker compose up --build -d
curl http://localhost:8080/health
docker compose down
```

5. Evidências de execução

A entrega inclui o roteiro docs/evidencias/README.md. Ele orienta a captura de cinco evidências reais: build/testes, publicação no GHCR, deploy em staging, aprovação/deploy em produção e healthcheck do Compose.

Evidência	Fonte	Situação desta versão
Build e testes	GitHub Actions	Workflow configurado; capturar após execução.
Imagem Docker	GHCR	Job configurado; capturar após publicação.
Staging	Azure App Service	Deploy automatizado; requer Web App e secrets.
Produção	Azure App Service	Deploy automatizado e protegido; requer aprovação.
Execução local	Docker Compose	Comandos documentados; executar no host com Docker.

Importante: não foram incluídos prints falsos. O material entregue contém a implementação e o roteiro para anexar os prints após a execução no ambiente do grupo.

6. Desafios encontrados e soluções

• Ausência de infraestrutura externa no momento da preparação: a solução foi parametrizada para receber nomes de Web Apps, publish profiles e conexão MongoDB por secrets/variáveis, sem expor credenciais.

• Necessidade de separar qualidade de promoção: o pipeline usa jobs dependentes e só publica imagem/deploy em branches de ambiente.

• Persistência em ambiente local: o Compose utiliza volumes nomeados, enquanto a aplicação recebe a conexão por variável de ambiente.

• Segurança mínima do container: a imagem de runtime utiliza o usuário não privilegiado app e não incorpora o arquivo .env.

7. Checklist de entrega

Item	OK	Observação
Projeto compactado em ZIP	☑	Pacote com estrutura organizada.
Dockerfile funcional	☑	Build multi-stage para .NET 8.
docker-compose.yml	☑	API + MongoDB, volumes, rede e healthcheck.
Pipeline build/test/deploy	☑	GitHub Actions para staging e produção.
README com instruções	☑	Inclui comandos, tecnologias e checklist.
Documentação técnica	☑	Este PDF.
Deploy realizado em staging e produção	☐	Pendente de execução nas contas externas; automação está pronta.

Conclusão

A entrega consolida os principais conceitos da fase DevOps: integração contínua, testes automatizados, empacotamento reproduzível, containerização, orquestração local e promoção controlada entre ambientes. Para concluir a última linha do checklist, basta conectar o repositório aos recursos Azure, cadastrar os secrets e executar o workflow nas branches qa e main.